

Hilbert Space Filling Curve-Style Theorem Search

A Python Reference Implementation, Mathematical Specification,
and Prior-Art Review

Version 0.4.0

August 6, 2026

By ChatGPT, inspired by Brian Tenneson

Executable status: the carried-forward Version 0.3.0 theorem-search implementation retains its 12-test passing result; Version 0.4.0 changes document pagination only.

Research status: corrected technical and historical edition; tested forward-saturation prototype; not yet a competitive ATP result.

Version 0.4.0 correction: pagination is normalized so the title page counts as physical page 1; table-of-contents numbers, printed footer numbers, and PDF-viewer page numbers now agree.

Contents

Abstract	4
The 135-Year Jigsaw: Where This Paper Fits	4
Corrections and scope clarifications in Version 0.3.0	5
1. Prior-art search result: no direct precedent located	6
2. The construction in one sentence	7
3. Mathematical foundation	7
3.1 Tenneson's explicit enumeration of the rationals	7
3.2 From $\mathbb{Q}$ to $\mathbb{Q} \cap [0, 1]$	8
3.3 One line number for a rational n-cube point	8
3.4 WFF line numbers as distinct rational points	8
4. Why the search space is a coproduct	9
5. Finite Hilbert approximants	9
6. Straight motion, turning, and changing rules	11
7. The probe and theorem-points	11
8. Frozen epochs and completeness	12
9. Budgets and interruption certificates	12
10. Training and post-training theorem discovery	12
11. Executable status and correctness checks	13
12. How to run the prototype	13
13. Sample verified output	13
14. What the prototype does not yet prove	14
15. Recommended next experiment	14

16. A Hilbertian epilogue	14
17. Objective assessment of the code	15
18. Objective assessment of the paper	15
19. Coproduct refresher	16
References	16
Appendix A. Complete Python reference implementation	18
Appendix B. Automated tests	33
Appendix C. Build, verification, and stress-test note	36
Section X	37
Conjectures	37
Theorems	37

Abstract

This document specifies and implements a budgeted theorem-discovery method in which finite arbitrary-dimensional Hilbert-style traversals order candidate inference applications. For a frozen verified theorem set C and a k -ary rule, the executable traverses a power-of-two grid of ranks in the frozen theorem list, which is sorted by WFF line number. Different inference rules live in a tagged coproduct, so arguments belonging to unrelated rules are not forced into one meaningless Cartesian product. Every accepted conclusion is stored with proof provenance and must pass a separate recomputing verification layer.

The document also develops an explicit rational-address layer from Brian Tenneson's surjection from the positive natural numbers onto the rationals, its least-preimage section, recursive tuple pairing, and a derived line numbering of rational n -cubes. A crucial correction in Version 0.3.0 is that these rational coordinates are global addresses and provenance in the current code; they do not yet determine the Hilbert metric. The implementation is a tested research prototype and forward consequence enumerator, not a decision procedure for ZFC and not yet a production Metamath prover.

The 135-Year Jigsaw: Where This Paper Fits

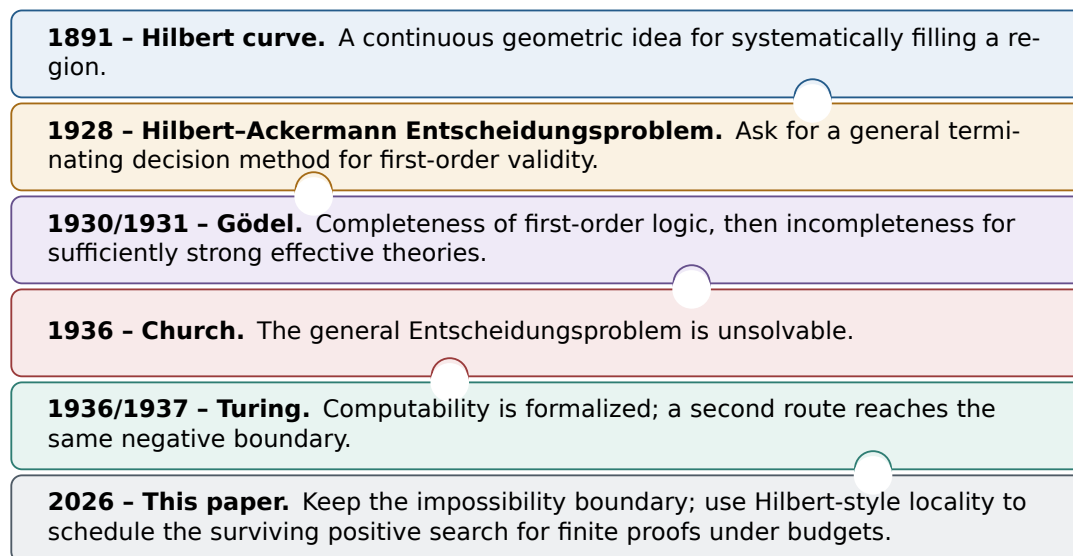


Figure 1: The historical jigsaw. The 2026 piece does not undo Gödel, Church, or Turing; it occupies the positive-search side that remains computably meaningful.

1891 - Hilbert curve. Hilbert published “Ueber die stetige Abbildung einer Linie auf ein Flaechenstueck” in Mathematische Annalen 38, pp. 459-460. The original construction is two-dimensional; the arbitrary-dimensional Hilbert traversals used in this paper are later generalizations inspired by Hilbert, not a construction Hilbert himself wrote down. [12]

1928 - Entscheidungsproblem. Hilbert and Ackermann’s Grundzuege der theoretischen Logik formulated the decision problem for first-order logic: is there a general effective procedure that determines whether a given formula is provable / logically valid? This

was not specifically a request for a ZFC theorem decider. [16]

1930-1931 - Goedel. Goedel's 1930 completeness theorem showed that every logically valid first-order formula is provable in a complete proof calculus; this is a positive proof-enumeration result, not a terminating decision algorithm. His 1931 incompleteness theorems then showed that sufficiently strong, consistent, effectively axiomatized theories containing arithmetic cannot settle every sentence and cannot, under the usual hypotheses, prove their own consistency. His 1931 paper explicitly discusses Principia Mathematica and Zermelo-Fraenkel-style set theory. [17,18]

1936 - Church. Church proved that the general Entscheidungsproblem is unsolvable: there is no effective procedure that always terminates with the correct yes/no answer for arbitrary first-order validity. [14,19]

1936/1937 - Turing. Turing independently made "effective procedure" mathematically precise with his machine model and again obtained a negative solution to the Entscheidungsproblem. The work was written in 1936 and published in the Proceedings of the London Mathematical Society in 1937. [15,20]

What remains. For ordinary effective proof calculi, theoremhood remains positively enumerable (semi-decidable): proofs are finite objects, so a fair enumeration can eventually find a proof that exists. What cannot be demanded in general is a finite stopping time that correctly declares every non-theorem. This paper is about scheduling that surviving positive search problem under budgets.

Do the 2026 arithmetic. The Hilbert curve is 135 years old (2026-1891). The Hilbert-Ackermann Entscheidungsproblem formulation is 98 years old (2026-1928). Goedel incompleteness is 95 years old (2026-1931). Church's and Turing's decisive computability results are 90 years old measured from 1936 (2026-1936).

Corrections and scope clarifications in Version 0.3.0

- The continuous limiting Hilbert curve is onto the full cube, hence onto every rational point in it. The executable algorithm, however, uses finite discrete Hilbert approximants. It visits grid cells exactly; it does not literally wait to hit arbitrary rational points of the limiting curve.
- Theorem identity, proof identity, and inference address are different objects. A WFF may be assigned a rational coordinate, and each candidate k-premise inference has a k-dimensional rational address, but a theorem can have many proofs and many inference histories. Trusted theorem identity remains the WFF line plus a verified certificate/provenance record.
- The current Version 0.3.0 Hilbert scheduler traverses ranks in the frozen theorem list. Tenneson rational n-cube coordinates are recorded as global addresses/provenance; they are not yet the metric that determines Hilbert adjacency. This is the most important implementation-level correction to the earlier exposition.
- For a formal system whose rules are actual computable partial functions of k explicit premises, rule arity k supplies the action dimension. For Metamath-style schemas, unification, substitutions, side conditions, and other independently variable choices may require additional coordinates. Arity alone is not automatically sufficient in a

production ATP.

- The toy verifier is separate from the search scheduler and recomputes rule applications, but it shares the same rule implementation. Calling it an “independent verifier” in the strong production-kernel sense was too strong. Version 0.3.0 calls it a separate recomputing verification layer.
- The completeness statement is a forward-saturation result under finite-rule, effective, fair-enumeration hypotheses. It is not a total decision procedure and does not by itself establish completeness of a future Metamath adapter.

1. Prior-art search result: no direct precedent located

A targeted literature search performed through August 6, 2026 found no published paper that directly uses a Hilbert space-filling curve or finite Hilbert order as the search scheduler over proof-theoretic inference addresses in automated theorem proving. Searches included combinations of “Hilbert curve,” “space-filling curve,” “proof search,” “automated theorem proving,” “automated reasoning,” “proof theory,” SAT, SMT, and formal verification across general web search and scholarly sources. This is a negative search result, not proof that no obscure precedent exists.

The neighboring literatures are real but distinct. Hilbert orders are used for locality-preserving multidimensional indexing, data mining, rendering, robotics, coverage, and other search/linearization problems [5,6,8,9,21]. Modern ATP work uses machine-learned ranking, Monte Carlo tree search, proof-structure enumeration, lemma generation, and many other search orderings [22,23]. In 2026, Bryant et al. reported an Isabelle/HOL autoformalization of Munkres topology that includes a space-filling-curve theorem, but the curve is something proved inside the formalization, not the mechanism used to search for proofs [10]. The careful novelty statement is therefore: to the best of the targeted search reported here, no direct precedent was located for Hilbert-order scheduling of ATP inference-address spaces.

Literature area	What exists	Relation to this project
Arbitrary-dimensional Hilbert algorithms	Butz, Skilling, and Haverkort give algorithms or systematic traversals for Hilbert-type curves in arbitrary finite dimension.	Directly supports the k -dimensional rule-component cursor. It does not supply an ATP interpretation.
Robotic coverage and exploration	Hilbert and other space-filling curves have been used to cover physical regions, including online obstacle avoidance.	Closest operational analogy: a path orders exhaustive visits. The searched objects here are premise tuples, not physical cells.
Multidimensional indexing and sampling	Space-filling curves linearize high-dimensional data while retaining locality approximately.	Supports the idea of reducing a multidimensional inference-address space to a one-dimensional schedule.
Formalization of space-filling curves	The space-filling-curve theorem itself has been formalized in proof assistants as part of larger topology developments.	The curve is the object being proved about, not the mechanism used to search for proofs.

Literature area	What exists	Relation to this project
Theorem provers named “Hilbert”	A 2025 agentic Lean framework is named Hilbert and recursively decomposes goals.	The name is unrelated to Hilbert space-filling curves; it is not prior art for this scheduling construction.
Direct ATP/proof-theory use located	No publication was located that uses a Hilbert/space-filling curve to enumerate tagged inference-rule premise tuples or to certify completed proof-search coverage.	This is a literature-search result, not a proof of novelty. A formal novelty claim requires a broader systematic search and expert review.

2. The construction in one sentence

Use a finite k-dimensional Hilbert order to enumerate every ordered k-tuple of currently verified theorems for each k-ary inference rule; keep different rules in a tagged coproduct so their arguments do not form meaningless Cartesian products; verify every new conclusion; repeat with the enlarged theorem set under an explicit expansion budget.

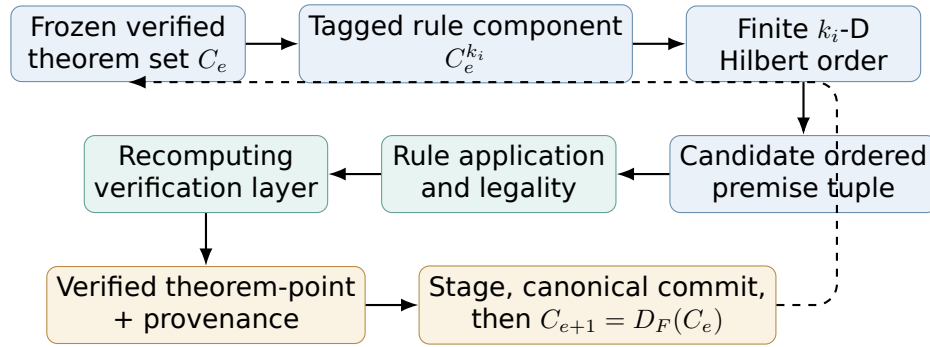


Figure 2: One frozen theorem-discovery epoch. Hilbert order schedules candidate inference addresses; verification alone licenses theorem-points.

3. Mathematical foundation

3.1 Tenneson’s explicit enumeration of the rationals

Let $\langle x \rangle = \lfloor x + \frac{1}{2} \rfloor$. Tenneson’s formulas for the two coordinates of the inverse Cantor-style pairing are, with $t = \langle \sqrt{2m} \rangle$,

$$A002260(m) = \frac{2m + t - t^2}{2}, \quad A004736(m) = \frac{2 - 2m + t + t^2}{2}.$$

His surjection $r : \mathbb{N}_+ \twoheadrightarrow \mathbb{Q}$ is

$$r(n) = (-1)^n \frac{A002260(\lfloor n/2 \rfloor)}{A004736(\lfloor n/2 \rfloor)}.$$

For a reduced rational p/q with $q \geq 1$, the least-preimage section s satisfies

$$r\left(s\left(\frac{p}{q}\right)\right) = \frac{p}{q}.$$

The implementation uses the two polynomial branches proved in Tenneson's paper.

3.2 From $\mathbb{Q}$ to $\mathbb{Q} \cap [0, 1]$

A useful explicit transport preserves rationality and produces a canonical least line. Reserve line 1 for the endpoint 1. For $n \geq 2$, set

$$u(1) = 1, \quad u(n) = \frac{|r(n-1)|}{1 + |r(n-1)|}.$$

Then $u : \mathbb{N}_+ \rightarrow \mathbb{Q} \cap [0, 1]$. For $0 \leq q < 1$, solve $q = t/(1+t)$ as $t = q/(1-q)$ and define

$$u_{\min}^{-1}(q) = s\left(\frac{q}{1-q}\right) + 1, \quad u_{\min}^{-1}(1) = 1.$$

Because Tenneson's positive occurrence of a nonzero magnitude precedes its negative occurrence, this section returns the least line of the transported unit-interval enumeration.

3.3 One line number for a rational n-cube point

Use Tenneson's pairing polynomial

$$\pi(a, b) = \frac{(a+b-1)(a+b-2)}{2} + a.$$

Recursively pairing n positive indices produces a bijection

$$\pi_n : \mathbb{N}_+^n \rightarrow \mathbb{N}_+.$$

Decoding one line and applying u coordinatewise gives an explicit surjection

$$R_n : \mathbb{N}_+ \rightarrow (\mathbb{Q} \cap [0, 1])^n.$$

Applying the unit-interval section coordinatewise and pairing the resulting indices gives a right inverse S_n , so

$$R_n(S_n(q_1, \dots, q_n)) = (q_1, \dots, q_n).$$

3.4 WFF line numbers as distinct rational points

A merely surjective map may assign several formulas to the same rational coordinate. Formula identity must therefore remain the WFF/Gödel line number. For geometry, use the injective map:

$$\iota(g) = \frac{g}{g+1}, \quad g = 0, 1, 2, \dots$$

An ordered premise tuple with WFF lines $(g_1, \dots, g_k)$ may be represented by the rational point $(\iota(g_1), \dots, \iota(g_k))$, and Tenneson's least-preimage machinery can assign a canonical global natural-number line to that rational tuple. In Version 0.3.0 this is an address/provenance layer. The actual finite Hilbert traversal operates on the rank tuple $(j_1, \dots, j_k)$ of those formulas in the WFF-line-sorted frozen theorem list. A future experiment may test Hilbert geometry built directly from a more semantically meaningful rational embedding.

4. Why the search space is a coproduct

Let the rules be $r_1, \dots, r_m$ with arities $k_1, \dots, k_m$. For a frozen theorem set C , the intrinsic inference-address space is:

$$\mathcal{A}_F(C) = \bigsqcup_{i=1}^m C^{k_i}.$$

A point is not an untagged tuple from one enormous product. It is a tagged object $(i; a_1, \dots, a_{k_i})$ saying exactly which rule and which ordered premises are being tested. The optional ambient orthogonal dimension is $K = \sum_i k_i$. K is useful for a geometric embedding, but the engine traverses each rule component at its own intrinsic dimension.

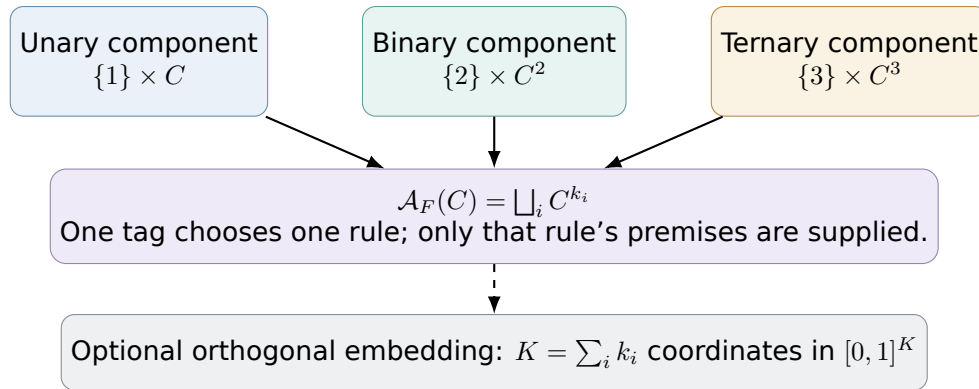


Figure 3: The coproduct keeps rule-specific argument coordinates from interfering. The sum of arities is an ambient embedding dimension, not the intrinsic dimension of every inference.

5. Finite Hilbert approximants

At the start of epoch e , freeze the verified theorem list $C_e = (\text{theta}_0, \dots, \text{theta}_{(M-1)})$, sorted by WFF line number. Choose $p = \text{ceil}(\log_2 M)$ and $N = 2^p$. A rule of arity k receives a discrete k -dimensional Hilbert-style bijection. Hilbert's 1891 paper described the two-dimensional curve; arbitrary-dimensional discrete algorithms are later generalizations, including work by Butz, Skilling, and others [3-6].

$$H_{k,p} : \{0, \dots, N^k - 1\} \longrightarrow \{0, \dots, N - 1\}^k.$$

A Hilbert step h decodes to a coordinate tuple $(j_1, \dots, j_k)$. If every $j_t < M$, it denotes the ordered premise tuple $(\text{theta}_{(j_1)}, \dots, \text{theta}_{(j_k)})$. Coordinates at least M are padding. Padding allows an exact power-of-two Hilbert grid without duplicating valid tuples. Every valid ordered tuple in C_e^k is visited exactly once in a completed rule-component pass.

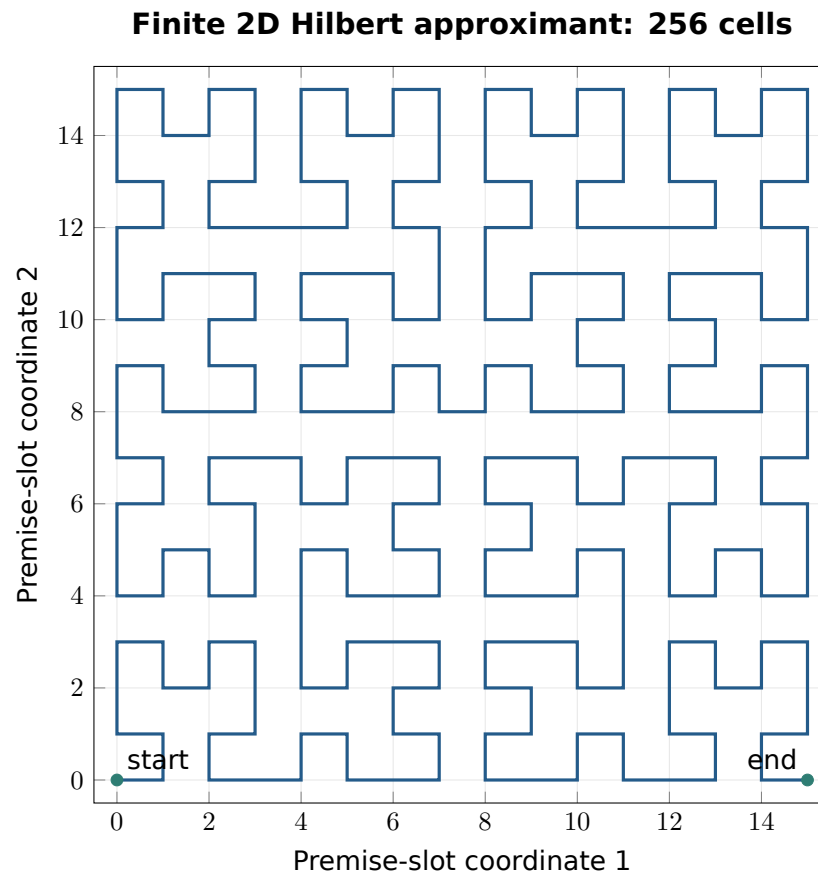


Figure 4: Vector-rendered two-dimensional finite Hilbert traversal. Consecutive cells are face-adjacent, so a local move changes exactly one premise-slot rank by one unit.

Finite 3D Hilbert approximant: 64 cells

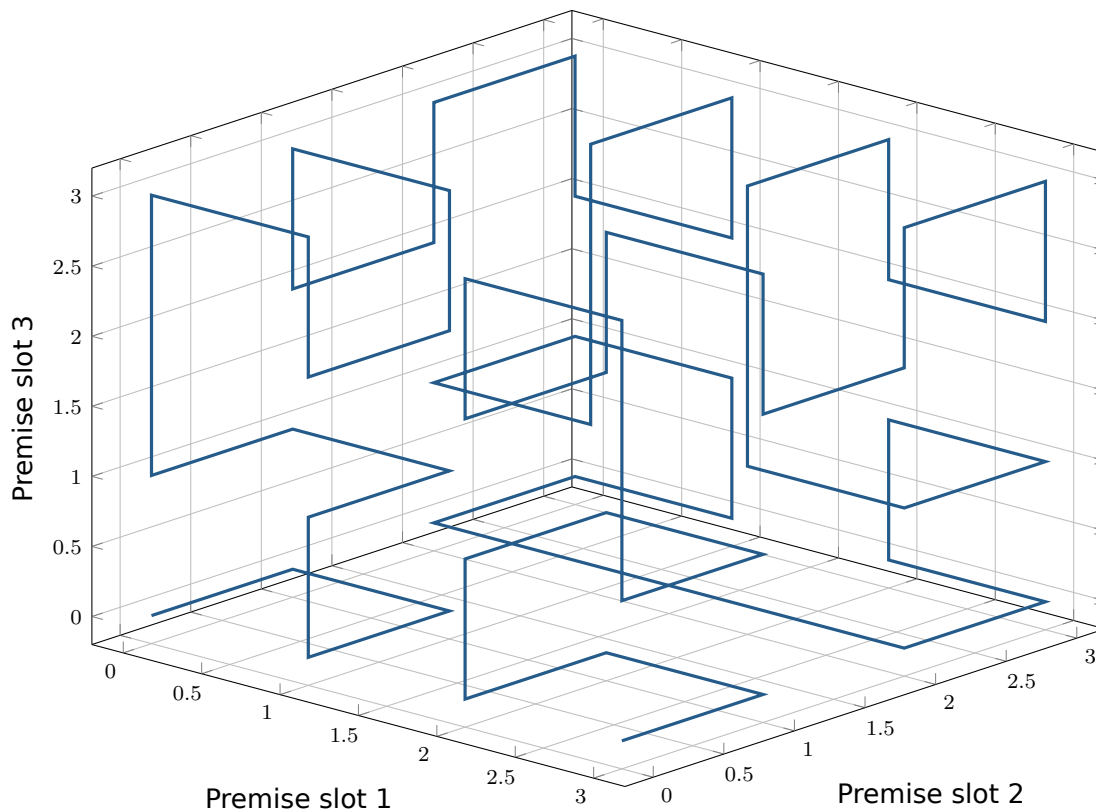


Figure 5: Vector-rendered three-dimensional finite Hilbert traversal for a ternary rule component.

6. Straight motion, turning, and changing rules

The discrete Hilbert path gives the geometric language an exact computational meaning. Straight motion means consecutive Hilbert steps continue to change the same coordinate axis: the same premise slot is varied while the other slots remain locally fixed. A turn within one rule component means the changed axis switches, so a different premise position is varied. A coproduct turn means the scheduler changes to a different tagged inference rule. In dimension k there are 2^k signed axis directions; excluding immediate reversal leaves $2^k - 1$ local nonreverse choices.

7. The probe and theorem-points

The probe is not merely the current Hilbert coordinate. It is the complete monotone verified search state: theorem database, proof/provenance forest, unfinished rule cursors, current resolution, Hilbert step, remaining budgets, and append-only event history. For a particular theorem, the relevant proof is the predecessor subgraph inside that forest.

A candidate formula becomes a trusted theorem record only after a proof certificate is accepted by the recomputing verifier layer. The stored object includes the WFF line, formula, certificate, rule name, premise lines, discovery epoch, and provenance address. Because the prototype verifier reuses the same rule implementation as the formal-system object, it is a

separation-of-responsibilities check rather than a fully independent production kernel.

8. Frozen epochs and completeness

Define the direct-consequence operator:

$$D_F(C) = C \cup \{r_i(a_1, \dots, a_{k_i}) : (a_1, \dots, a_{k_i}) \in C^{k_i}, r_i \text{ is defined}\}.$$

During one epoch, every premise comes from the same frozen C_e . Novel verified conclusions are staged and committed only after the pass ends. If every rule component completes, every rule is a computable finite-arity partial function on explicit premises, and the verifier accepts exactly the legal consequences, then $C_{(e+1)} = D_F(C_e)$. Repeated fair complete epochs enumerate the deductive closure generated by those rules. This does not yet cover a real Metamath search until independently variable substitutions, unification choices, and side conditions have also been made part of the enumerable action space.

9. Budgets and interruption certificates

One expansion is one attempted inference-rule application on one valid decoded premise tuple. Address visits are counted separately because a padded power-of-two grid may include cells that do not decode to current theorem indices. For M frozen theorems, the exact valid expansion count of a completed epoch is:

$$E(M) = \sum_i M^{k_i}.$$

With $N = 2^{\lceil \log_2 M \rceil}$, the total Hilbert address visits are:

$$A(M) = \sum_i N^{k_i}.$$

Padding overhead bound. For $M > 1$, $M \leq N < 2M$. Thus a k -ary component visits $N^k < 2^k M^k$ grid cells. The power-of-two embedding can therefore cost less than a factor 2^k in address visits relative to the exact M^k valid tuple count; it does not reduce the exhaustive combinatorial cost.

The implementation records cells visited, valid tuples attempted, padding visits, verifier calls, new theorem-points, the exact cursor distance for each component, and whether the epoch completed. An interrupted run therefore reports bounded failure and measured coverage; it never converts silence into refutation or independence.

10. Training and post-training theorem discovery

After training, the system's operational purpose is theorem discovery. A frozen learned policy may choose the order of rule components, starting orientation, subcube priority, exploration temperature, or budget allocation. The wrapper retains a fair fallback schedule and may require eventual completion of all cells for any run advertised as exhaustive. The learned model changes navigation; the formal rules determine legality; the verifier determines theoremhood.

11. Executable status and correctness checks

The supplied Python program and tests were executed in the build environment. All 12 tests passed. The checks include:

- Tenneson's r and s satisfy $r(s(q)) = q$ on representative positive, negative, zero, and fractional inputs, and s returns the least tested preimage.
- The pairing polynomial and recursive tuple coding round-trip exactly.
- The derived unit-interval map and rational n -cube line numbering round-trip exactly.
- The WFF coordinate $\text{iota}(g)$ is injective and invertible on its image.
- Hilbert index-to-point and point-to-index maps are bijective for dimensions 1 through 5 and several resolutions.
- Consecutive Hilbert cells differ by Manhattan distance one.
- A complete Hilbert epoch attempts every valid ordered premise tuple exactly once.
- A complete Hilbert epoch equals an independently implemented exhaustive direct consequence layer.
- A partial expansion budget produces an incomplete but internally consistent coverage report.
- The separate recomputing verifier layer rejects a deliberately tampered certificate.
- Repeated epochs reach the entire finite closure of the toy system.
- The append-only transaction hash chain verifies successfully.

12. How to run the prototype

Place the two Python files in the same directory and run:

```
python -m unittest -v test_hilbert_theorem_search.py
```

```
python hilbert_theorem_search.py
```

The demonstration formal system has one unary, one binary, and one ternary rule, so its ambient orthogonal dimension is $K = 1 + 2 + 3 = 6$. It discovers every bounded theorem $P(0)$ through $P(12)$, reports exact expansion and padding counts for each epoch, and verifies the hash chain.

13. Sample verified output

Hilbert Space Filling Curve-Style Theorem Search

By ChatGPT, inspired by Brian Tenneson

Formal system: Bounded toy arithmetic through 12

Ambient orthogonal dimension $K = 6$

epoch=1 frozen=2 expansions=14 new=5 complete=True

epoch=2 frozen=7 expansions=399 new=6 complete=True

epoch=3 frozen=13 expansions=2379 new=0 complete=True

Verified theorem database:

P(0), P(1), P(2), P(3), P(4), P(5), P(6), P(7), P(8), P(9), P(10), P(11), P(12)

Hash chain valid: True

14. What the prototype does not yet prove

- It does not establish that Hilbert ordering outperforms breadth-first, best-first, random, learned, or portfolio ATP baselines.
- It does not show that geometric locality in premise-index space correlates with logical usefulness in a large proof library.
- It does not yet connect to Metamath set.mm, Lean, Isabelle, or a production verifier.
- It does not turn an incomplete budgeted run into a negative theoremhood result.
- It does not establish a publication level novelty claim merely from the literature searches reported here.
- The rational geometry is an explicit addressing and ordering framework; empirical work must test which formula coding makes locality useful.
- The Tenneson rational address is currently logged but does not determine Hilbert distance; the empirical value of a rational theorem geometry remains untested.
- Formula-line locality is not automatically semantic locality. If nearby WFF lines are logically unrelated, Hilbert locality can preserve the wrong neighborhood structure.
- The toy verifier is not an independently implemented Metamath-style kernel; it shares the formal-system rule implementation.
- Rule arity alone is insufficient for assertion schemas with independently variable substitutions or unification choices; those degrees of freedom must be enumerated in a real adapter.
- A complete Hilbert sweep has the same valid-tuple count as exhaustive enumeration. Any practical advantage must come from better ordering under interruption, better locality, learned allocation, or reduced overhead in downstream operations - not from magically changing the size of the complete search space.

15. Recommended next experiment

Connect the engine to a small frozen Metamath database or a deliberately tiny Hilbert-style propositional calculus. Compare four matched-budget schedules: lexicographic tuple order, random order, breadth-balanced rule order, and Hilbert order. Measure verified theorem discoveries, duplicate rate, expansions, verifier calls, proof length, wall time, memory, and the fraction of each rule component completed. Then add a trained policy that may reorder components but cannot change the complete fallback schedule. The crucial empirical question is not whether the Hilbert curve is onto; it is whether locality under the chosen WFF/premise encoding improves useful theorem discovery.

16. A Hilbertian epilogue

No one can know how David Hilbert would have judged this specific construction. The historically responsible observation is that two different Hilbert legacies meet here: the 1891 space-filling curve supplies a systematic locality-preserving traversal idea, while the later Hilbert program and

Entscheidungsproblem made mechanical proof and decision central foundational questions. The 2026 construction borrows the former without pretending to solve the latter.

Goedel, Church, and Turing force a disciplined target. There is no universal terminating algorithm that decides arbitrary first-order theoremhood/validity in the way Hilbert and Ackermann hoped, and sufficiently strong effective theories such as standard set theories cannot be complete. What remains is positive proof search: enumerate legal finite proof objects, improve the order in which they are explored, certify every success, and label finite failure UNKNOWN.

In that limited sense, this paper can be read as a next-best Hilbertian engineering question: once total decision is unavailable, can a space-filling-curve-style schedule make bounded positive theorem search more systematic, more interruption-aware, and empirically more productive? That is a falsifiable algorithmic question, not a claim to evade undecidability.

17. Objective assessment of the code

What is solid. The Hilbert index/point conversion is genuinely arbitrary-dimensional in the tested range; bijection and face-adjacency properties pass both the 12-unit-test suite and additional stress checks. The Tenneson r/s and rational-address round trips were also stress-tested on thousands of outputs and small rational grids. Frozen epochs agree with an independently written exhaustive direct-consequence baseline on the toy system.

What is only prototype-grade. The engine has no production parser, unifier, substitution enumerator, term index, parallel work queue, checkpoint/resume layer, large-library memory strategy, or real proof-kernel integration. Its verifier is separate but not independently implemented. The current geometry is over ranks in a frozen theorem list; the rational address layer is meta-data/provenance rather than the actual search metric.

Engineering judgment. As a research reference implementation, the code is coherent, auditable, and suitable for controlled pilot experiments. As a competitive ATP, it is not ready. The next meaningful milestone is not more toy features; it is an adapter to a real proof environment with matched-budget controls and externally verified certificates.

18. Objective assessment of the paper

Strongest contribution. The paper now has a clear algorithmic thesis: represent rule applications as a coproduct of variable-dimensional action spaces and use finite Hilbert-style orders to schedule exhaustive positive proof search under interruption. The combination with explicit rational line addressing is unusual and conceptually clean as an address/provenance layer.

What is not new by itself. Hilbert curves, arbitrary-dimensional Hilbert orders, coproducts/disjoint unions, proof enumeration, forward saturation, and semi-decidability are established ideas. The completeness proof for completed finite frozen epochs is essentially the standard fact that exhaustive fair application of effective finitary inference rules enumerates deductive closure. Novelty must therefore live in the specific synthesis and, ultimately, in demonstrated search behavior.

Publication status. In its present corrected form, the paper is credible as a research note, technical report, preprint, or workshop-position paper. A stronger peer-reviewed ATP paper would need a real prover integration, a frozen benchmark protocol, multiple targets, and matched controls such as lexicographic, random, breadth/best-first, and learned policies.

Main scientific risk. Hilbert locality is useful only if nearby coordinates correspond to proof-relevant similarity. Arbitrary Godel/WFF numbering may destroy that relationship. The strongest next hypothesis is therefore not merely “Hilbert beats random,” but “Hilbert ordering over a struc-

turally or learned semantically ordered formula space improves verified theorem yield per expansion under fixed budgets.”

Novelty claim. The targeted literature search found no direct precedent for using Hilbert space-filling-curve order as the scheduler over ATP inference-address spaces. That is encouraging but not conclusive; a formal novelty claim should remain qualified until a broader bibliographic review and peer scrutiny are complete.

19. Coproduct refresher

For sets. The coproduct A disjoint-union B is the tagged disjoint union $(\{0\} \times A) \cup (\{1\} \times B)$. The tag remembers which component an element came from even if A and B overlap. A Cartesian product $A \times B$ instead chooses an element of A and an element of B simultaneously.

Why it fits inference rules. One proof step chooses one inference rule and supplies the arguments for that rule. It does not simultaneously supply arguments to every rule in the formal system. Therefore the intrinsic action space is a coproduct of rule-specific products $C^{(k_i)}$, not the Cartesian product of all rule coordinates at once.

When do coproducts and products coincide? In **Set** they generally do not. Trivial simplifications include A disjoint-union empty-set isomorphic to A and $A \times \{\text{star}\}$ isomorphic to A . Finite sets can have accidental equal cardinality - for nonempty finite A and B , $|A| + |B| = |A||B|$ only in the 2-by-2 case - but that does not give a natural structural identification. For infinite cardinalities, sums and products can also have the same size without being canonically the same object.

A real categorical coincidence. In categories with finite biproducts - for example vector spaces, abelian groups, and modules - finite products and finite coproducts are naturally isomorphic and are written as direct sums. Infinite direct products and direct sums generally differ. The proof-search construction here is **Set**-like and uses tagged disjoint unions, so the biproduct coincidence does not collapse the rule coproduct into one Cartesian product.

Distributivity. Products can distribute over coproducts: $A \times (B \sqcup C) \cong (A \times B) \sqcup (A \times C)$ in **Set**. This can simplify mixed expressions, but it still does not identify coproduct with product.

References

- [1] Brian Tenneson. A Closed-Form Surjection from N onto Q — With Its Pseudo-Inverse. Version 2.2, July 17, 2026.
- [2] Brian Tenneson. The Universal Proof-Horizon Operator: From MIU and the n th Well-Formed Formula to ZFC, Predator, and Automated Logical Deciders. Version 1.0, August 3, 2026.
- [3] Arthur R. Butz. Convergence with Hilbert’s Space Filling Curve. *Journal of Computer and System Sciences* 3(2), 1969. DOI: 10.1016/S0022-0000(69)80010-3. [https://doi.org/10.1016/S0022-0000\(69\)80010-3](https://doi.org/10.1016/S0022-0000(69)80010-3)
- [4] Arthur R. Butz. Alternative Algorithm for Hilbert’s Space-Filling Curve. *IEEE Transactions on Computers* C-20(4), 424–426, 1971. DOI: 10.1109/T-C.1971.223258. <https://doi.org/10.1109/T-C.1971.223258>
- [5] John Skilling. Programming the Hilbert Curve. *AIP Conference Proceedings* 707, 381–387, 2004. DOI: 10.1063/1.1751381. <https://doi.org/10.1063/1.1751381>
- [6] Herman Haverkort. Sixteen Space-Filling Curves and Traversals for d -Dimensional Cubes and Simplices. arXiv:1711.04473. <https://arxiv.org/abs/1711.04473>
- [7] Anant A. Joshi, Maulik C. Bhatt, and Arpita Sinha. Modification of Hilbert’s Space-Filling Curve

- to Avoid Obstacles: A Robotic Path-Planning Strategy. Indian Control Conference, 2019. DOI: 10.1109/ICC47138.2019.9123166. <https://doi.org/10.1109/ICC47138.2019.9123166>
- [8] Ashay Wakode and Arpita Sinha. Space Filling Curves for Coverage Path Planning with Online Obstacle Avoidance. arXiv:2209.01426. <https://arxiv.org/abs/2209.01426>
- [9] Christian Böhm. Space-Filling Curves for High-Performance Data Mining. arXiv:2008.01684. <https://arxiv.org/abs/2008.01684>
- [10] Dustin Bryant, Jonathan Julián Huerta y Munive, Cezary Kaliszyk, and Josef Urban. Munkres' General Topology Autoformalized in Isabelle/HOL. arXiv:2604.07455, 2026. <https://arxiv.org/abs/2604.07455>
- [11] Sumanth Varambally et al. Hilbert: Recursively Building Formal Proofs with Informal Reasoning. arXiv:2509.22819, 2025. This "Hilbert" is unrelated to the space-filling-curve scheduler. <https://arxiv.org/abs/2509.22819>
- [12] David Hilbert. Ueber die stetige Abbildung einer Linie auf ein Flaechenstueck. Mathematische Annalen 38 (1891), 459-460.
- [13] David Hilbert. Mathematical Problems. Lecture delivered before the International Congress of Mathematicians, Paris, 1900; English translation published 1902.
- [14] Alonzo Church. An Unsolvable Problem of Elementary Number Theory. American Journal of Mathematics 58(2), 345-363, 1936.
- [15] A. M. Turing. On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, Series 2, 42, 230-265, 1936/1937.
- [16] David Hilbert and Wilhelm Ackermann. Grundzuege der theoretischen Logik. Springer, Berlin, 1928.
- [17] Kurt Goedel. Die Vollstaendigkeit der Axiome des logischen Funktionenkalkuels. Monatshefte fuer Mathematik und Physik 37, 349-360, 1930.
- [18] Kurt Goedel. Ueber formal unentscheidbare Saetze der Principia Mathematica und verwandter Systeme I. Monatshefte fuer Mathematik und Physik 38, 173-198, 1931. DOI: 10.1007/BF01700692.
- [19] Alonzo Church. A Note on the Entscheidungsproblem. Journal of Symbolic Logic 1(1), 40-41, 1936; correction 1(3), 101-102, 1936. DOI: 10.2307/2269326.
- [20] Stanford Encyclopedia of Philosophy. The Church-Turing Thesis: The Rise and Fall of the Entscheidungsproblem; Alan Turing; Goedel's Incompleteness Theorems. Accessed August 6, 2026.
- [21] C. Gotsman and M. Lindenbaum. On the Metric Properties of Discrete Space-Filling Curves. IEEE Transactions on Image Processing 5(5), 794-797, 1996. DOI: 10.1109/83.499920.
- [22] Michael Faerber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. Journal of Automated Reasoning 65, 287-320, 2021. DOI: 10.1007/s10817-020-09576-7.
- [23] Christoph Wernhard and Wolfgang Bibel. Investigations into Proof Structures. Journal of Automated Reasoning 68, Article 24, 2024. DOI: 10.1007/s10817-024-09711-8.

Appendix A. Complete Python reference implementation

The complete reference implementation carried forward into document Version 0.4.0 is reproduced below. The theorem-search code is unchanged from Version 0.3.0; Version 0.4.0 corrects the document pagination and does not alter the search algorithm.

```

1  #!/usr/bin/env python3
2  """Hilbert Space Filling Curve-Style Theorem Search.
3
4  By ChatGPT, inspired by Brian Tenneson.
5
6  This is a self-contained reference implementation of a budgeted theorem-
7  discovery engine. It combines:
8
9  1. Brian Tenneson's explicit surjection  $N^+ \rightarrow Q$  and least-preimage section;
10 2. an explicit line numbering of rational  $n$ -cubes;
113. injective rational coordinates for WFF/Godel line numbers;
124. arbitrary-dimensional discrete Hilbert traversal (Skilling-style);
135. a coproduct of rule-arity premise spaces;
146. frozen theorem-discovery epochs, proof certificates, and an independent
15  verifier;
167. expansion and address-visit budgets with partial coverage reports.
17
18  The executable toy formal system at the bottom has rules of arity 1, 2, and 3.
19  It is intentionally small enough to audit. Replacing it with Metamath, Lean,
20  or another formal system requires adapters for formula encoding, legal rule
21  application, and certificate verification; the Hilbert scheduler itself is
22  formal-system agnostic.
23  """
24
25  from __future__ import annotations
26
27  from dataclasses import asdict, dataclass, field
28  from fractions import Fraction
29  import hashlib
30  import json
31  import math
32  from typing import (
33      Callable,
34      Generic,
35      Hashable,
36      Iterable,
37      Iterator,
38      Mapping,
39      Protocol,
40      Sequence,
41      TypeVar,
42  )
43
44  FormulaT = TypeVar("FormulaT", bound=Hashable)
45
46
47  # -----
48  # Part I. Brian Tenneson's  $N^+ \rightarrow Q$  formulas and rational-cube line numbers
49  # -----
50
51
52  def _round_sqrt_2n(n: int) -> int:
53      """Return floor(sqrt(2n) + 1/2) using exact integer arithmetic."""
54      if n < 0:
55          raise ValueError("n must be nonnegative")
56      x = 2 * n
57      a = math.isqrt(x)
58      # sqrt(x) >= a + 1/2 iff 4x >= (2a+1)^2.
59      return a + (1 if 4 * x >= (2 * a + 1) ** 2 else 0)
60
61
62  def a002260(n: int) -> int:

```

```

63     """Tenneson's closed form for the Smarandache crescendo coordinate."""
64     t = _round_sqrt_2n(n)
65     return (2 * n + t - t * t) // 2
66
67
68 def a004736(n: int) -> int:
69     """Tenneson's closed form for the Smarandache decrescendo coordinate."""
70     t = _round_sqrt_2n(n)
71     return (2 - 2 * n + t + t * t) // 2
72
73
74 def tenneson_r(n: int) -> Fraction:
75     """Brian Tenneson's explicit surjection  $r: \mathbb{N}^+ \rightarrow \mathbb{Q}$ .
76
77      $r(n) = (-1)^n A002260(\text{floor}(n/2)) / A004736(\text{floor}(n/2))$ .
78     """
79     if n < 1:
80         raise ValueError("n must be a positive integer")
81     m = n // 2
82     numerator = a002260(m)
83     denominator = a004736(m)
84     sign = -1 if n % 2 else 1
85     return Fraction(sign * numerator, denominator)
86
87
88 def tenneson_s(value: Fraction | int) -> int:
89     """Return the least  $n \geq 1$  with  $\text{tenneson\_r}(n) = \text{value}$ .
90
91     The input Fraction is automatically reduced. This is the right inverse
92     (section) from Tenneson's paper:  $r(s(q)) = q$ .
93     """
94     q = Fraction(value)
95     p = q.numerator
96     d = q.denominator
97     if p >= 1:
98         return 2 - p + p * p - 3 * d + 2 * p * d + d * d
99     return 3 + p + p * p - 3 * d - 2 * p * d + d * d
100
101
102 def pair_positive(a: int, b: int) -> int:
103     """Tenneson's/Cantor-style bijection  $\mathbb{N}^+ \times \mathbb{N}^+ \rightarrow \mathbb{N}^+$ ."""
104     if a < 1 or b < 1:
105         raise ValueError("pair_positive requires positive integers")
106     return ((a + b - 1) * (a + b - 2)) // 2 + a
107
108
109 def unpair_positive(line: int) -> tuple[int, int]:
110     """Inverse of pair_positive using A002260 and A004736."""
111     if line < 1:
112         raise ValueError("line must be positive")
113     return a002260(line), a004736(line)
114
115
116 def pair_tuple(values: Sequence[int]) -> int:
117     """Recursively encode a nonempty positive-integer tuple as one line."""
118     if not values:
119         raise ValueError("cannot encode an empty tuple")
120     if any(value < 1 for value in values):
121         raise ValueError("tuple entries must be positive")
122     code = values[-1]
123     for value in reversed(values[:-1]):
124         code = pair_positive(value, code)
125     return code
126
127
128 def unpair_tuple(line: int, dimension: int) -> tuple[int, ...]:
129     """Decode one positive line into a positive-integer tuple of given length."""
130     if line < 1:
131         raise ValueError("line must be positive")
132     if dimension < 1:
133         raise ValueError("dimension must be positive")

```

```

134     if dimension == 1:
135         return (line,)
136     first, rest = unpair_positive(line)
137     return (first,) + unpair_tuple(rest, dimension - 1)
138
139
140 def unit_rational(line: int) -> Fraction:
141     """Explicit surjection  $\mathbb{N} \rightarrow \mathbb{Q} \cap [0,1]$ .
142
143     Line 1 is reserved for 1. All other lines transport Tenneson's rational
144     enumeration through  $|q|/(1+|q|)$ , which is rational and lies in  $[0,1]$ .
145     """
146     if line < 1:
147         raise ValueError("line must be positive")
148     if line == 1:
149         return Fraction(1, 1)
150     q = abs(tenneson_r(line - 1))
151     return q / (1 + q)
152
153
154 def unit_rational_section(value: Fraction | int) -> int:
155     """Canonical least line for a rational in  $[0,1]$ .
156
157     For  $0 \leq q < 1$ , invert  $q = t/(1+t)$  by  $t = q/(1-q)$ , then invoke
158     Tenneson's least-preimage section and shift by one.
159     """
160     q = Fraction(value)
161     if q < 0 or q > 1:
162         raise ValueError("value must lie in  $\mathbb{Q} \cap [0,1]$ ")
163     if q == 1:
164         return 1
165     t = q / (1 - q)
166     return tenneson_s(t) + 1
167
168
169 def rational_cube_point(line: int, dimension: int) -> tuple[Fraction, ...]:
170     """Decode one line into a point of  $(\mathbb{Q} \cap [0,1])^{\text{dimension}}$ ."""
171     indices = unpair_tuple(line, dimension)
172     return tuple(unit_rational(index) for index in indices)
173
174
175 def rational_cube_line(point: Sequence[Fraction | int]) -> int:
176     """Return a canonical line for a rational point in the unit n-cube."""
177     if not point:
178         raise ValueError("point must be nonempty")
179     return pair_tuple([unit_rational_section(Fraction(x)) for x in point])
180
181
182 def wff_coordinate(godel_line: int) -> Fraction:
183     """Injectively place a zero-based WFF/Godel line in  $\mathbb{Q} \cap [0,1]$ .
184
185      $\text{iota}(g) = g/(g+1)$ . Unlike a merely surjective enumeration, this map never
186     identifies two different WFF line numbers.
187     """
188     if godel_line < 0:
189         raise ValueError("godel_line must be nonnegative")
190     return Fraction(godel_line, godel_line + 1)
191
192
193 def wff_line_from_coordinate(value: Fraction | int) -> int:
194     """Inverse of wff_coordinate on its image."""
195     q = Fraction(value)
196     if q < 0 or q >= 1:
197         raise ValueError("WFF coordinate must lie in  $[0,1]$ ")
198     candidate = q / (1 - q)
199     if candidate.denominator != 1:
200         raise ValueError("coordinate is not in the image of wff_coordinate")
201     return candidate.numerator
202
203
204 def premise_point(godel_lines: Sequence[int]) -> tuple[Fraction, ...]:

```

```

205     """Rational point encoding an ordered tuple of WFF line numbers."""
206     if not godel_lines:
207         raise ValueError("premise tuple must be nonempty")
208     return tuple(wff_coordinate(line) for line in godel_lines)
209
210
211 def premise_point_line(godel_lines: Sequence[int]) -> int:
212     """Canonical global line of an ordered premise tuple."""
213     return rational_cube_line(premise_point(godel_lines))
214
215
216 def inference_address_line(rule_tag: int, premise_godel_lines: Sequence[int]) -> int:
217     """One global line for a tagged coproduct inference address.
218
219     rule_tag is zero-based. The +1 converts it to the positive convention used
220     by pair_positive.
221     """
222     if rule_tag < 0:
223         raise ValueError("rule_tag must be nonnegative")
224     return pair_positive(rule_tag + 1, premise_point_line(premise_godel_lines))
225
226
227 # -----
228 # Part II. Arbitrary-dimensional discrete Hilbert traversal
229 # -----
230
231
232 class HilbertCurve:
233     """Discrete Hilbert index <-> coordinate map in arbitrary dimension.
234
235     This implementation follows the transpose/Gray-code method associated with
236     John Skilling's "Programming the Hilbert curve". At `bits` bits per axis
237     and `dimension` axes it is a bijection between
238
239         {0, ..., 2^(bits*dimension)-1}
240
241     and
242
243         {0, ..., 2^bits-1}^dimension.
244     """
245
246     def __init__(self, dimension: int, bits: int) -> None:
247         if dimension < 1:
248             raise ValueError("dimension must be positive")
249         if bits < 0:
250             raise ValueError("bits must be nonnegative")
251         self.dimension = dimension
252         self.bits = bits
253         self.side = 1 << bits
254         self.size = 1 << (bits * dimension)
255
256     def _integer_to_transpose(self, distance: int) -> list[int]:
257         if self.bits == 0:
258             return [0] * self.dimension
259         bit_string = format(distance, f"0{self.bits * self.dimension}b")
260         return [
261             int(bit_string[axis :: self.dimension], 2)
262             for axis in range(self.dimension)
263         ]
264
265     def _transpose_to_integer(self, coordinates: Sequence[int]) -> int:
266         if self.bits == 0:
267             return 0
268         words = [format(value, f"0{self.bits}b") for value in coordinates]
269         interleaved = "".join(
270             words[axis][bit]
271             for bit in range(self.bits)
272             for axis in range(self.dimension)
273         )
274         return int(interleaved, 2)
275

```

```

276 def point_from_distance(self, distance: int) -> tuple[int, ...]:
277     """Decode a Hilbert step number into grid coordinates."""
278     if distance < 0 or distance >= self.size:
279         raise ValueError("distance must lie in [0, {self.size})")
280     if self.bits == 0:
281         return (0,) * self.dimension
282
283     x = self._integer_to_transpose(distance)
284
285     # Gray decode.
286     t = x[self.dimension - 1] >> 1
287     for axis in range(self.dimension - 1, 0, -1):
288         x[axis] ^= x[axis - 1]
289     x[0] ^= t
290
291     # Undo the dimension-dependent excess work.
292     q = 2
293     limit = 1 << self.bits
294     while q != limit:
295         p = q - 1
296         for axis in range(self.dimension - 1, -1, -1):
297             if x[axis] & q:
298                 x[0] ^= p
299             else:
300                 t = (x[0] ^ x[axis]) & p
301                 x[0] ^= t
302                 x[axis] ^= t
303         q <<= 1
304
305     return tuple(x)
306
307 def distance_from_point(self, point: Sequence[int]) -> int:
308     """Encode grid coordinates as their Hilbert step number."""
309     if len(point) != self.dimension:
310         raise ValueError("point has the wrong dimension")
311     if any(value < 0 or value >= self.side for value in point):
312         raise ValueError("coordinates must lie in [0, {self.side})")
313     if self.bits == 0:
314         return 0
315
316     x = list(point)
317     q = 1 << (self.bits - 1)
318
319     # Inverse of the final exchange/invert stage.
320     while q > 1:
321         p = q - 1
322         for axis in range(self.dimension):
323             if x[axis] & q:
324                 x[0] ^= p
325             else:
326                 t = (x[0] ^ x[axis]) & p
327                 x[0] ^= t
328                 x[axis] ^= t
329         q >>= 1
330
331     # Gray encode.
332     for axis in range(1, self.dimension):
333         x[axis] ^= x[axis - 1]
334     t = 0
335     q = 1 << (self.bits - 1)
336     while q > 1:
337         if x[self.dimension - 1] & q:
338             t ^= q - 1
339         q >>= 1
340     for axis in range(self.dimension):
341         x[axis] ^= t
342
343     return self._transpose_to_integer(x)
344
345 def points(self) -> Iterator[tuple[int, ...]]:
346     for distance in range(self.size):

```

```

347         yield self.point_from_distance(distance)
348
349
350 @dataclass(slots=True)
351 class HilbertCursor:
352     """Mutable cursor over one finite Hilbert approximant."""
353
354     dimension: int
355     bits: int
356     next_distance: int = 0
357     curve: HilbertCurve = field(init=False)
358
359     def __post_init__(self) -> None:
360         self.curve = HilbertCurve(self.dimension, self.bits)
361
362     @property
363     def done(self) -> bool:
364         return self.next_distance >= self.curve.size
365
366     @property
367     def total_cells(self) -> int:
368         return self.curve.size
369
370     def next_point(self) -> tuple[int, ...]:
371         if self.done:
372             raise StopIteration
373         point = self.curve.point_from_distance(self.next_distance)
374         self.next_distance += 1
375         return point
376
377
378 # -----
379 # Part III. Formal-system interfaces, certificates, and theorem storage
380 # -----
381
382
383 @dataclass(frozen=True, slots=True)
384 class InferenceRule(Generic[FormulaT]):
385     """A finitary inference rule.
386
387     `derive` may return zero, one, or several conclusions for one ordered
388     premise tuple. It must be deterministic for reproducible runs.
389     """
390
391     name: str
392     arity: int
393     derive: Callable[[tuple[FormulaT, ...]], Iterable[FormulaT]]
394
395     def __post_init__(self) -> None:
396         if self.arity < 0:
397             raise ValueError("rule arity must be nonnegative")
398
399
400 @dataclass(frozen=True, slots=True)
401 class FormalSystem(Generic[FormulaT]):
402     name: str
403     axioms: tuple[FormulaT, ...]
404     rules: tuple[InferenceRule[FormulaT], ...]
405     encode_formula: Callable[[FormulaT], int]
406     canonical_key: Callable[[FormulaT], object] = repr
407
408     def __post_init__(self) -> None:
409         names = [rule.name for rule in self.rules]
410         if len(names) != len(set(names)):
411             raise ValueError("rule names must be unique")
412         axiom_codes = [self.encode_formula(axiom) for axiom in self.axioms]
413         if any(code < 0 for code in axiom_codes):
414             raise ValueError("formula encodings must be nonnegative")
415         if len(axiom_codes) != len(set(axiom_codes)):
416             raise ValueError("axioms must have distinct formula encodings")
417

```

```

418     @property
419     def ambient_orthogonal_dimension(self) -> int:
420         return sum(rule.arity for rule in self.rules)
421
422     def rule_by_name(self, name: str) -> InferenceRule[FormulaT]:
423         for rule in self.rules:
424             if rule.name == name:
425                 return rule
426         raise KeyError(name)
427
428
429 @dataclass(frozen=True, slots=True)
430 class ProofCertificate(Generic[FormulaT]):
431     conclusion: FormulaT
432     conclusion_line: int
433     rule_name: str
434     premise_lines: tuple[int, ...]
435     premise_formulas: tuple[FormulaT, ...]
436     inference_line: int | None
437
438
439 @dataclass(frozen=True, slots=True)
440 class TheoremRecord(Generic[FormulaT]):
441     formula: FormulaT
442     wff_line: int
443     certificate: ProofCertificate[FormulaT]
444     discovery_epoch: int
445
446
447 class IndependentVerifier(Generic[FormulaT]):
448     """Recompute each claimed rule application outside the scheduler."""
449
450     AXIOM_RULE = "AXIOM"
451
452     def __init__(self, system: FormalSystem[FormulaT]) -> None:
453         self.system = system
454         self._axioms = set(system.axioms)
455
456     def verify(
457         self,
458         certificate: ProofCertificate[FormulaT],
459         known_by_line: Mapping[int, TheoremRecord[FormulaT]],
460     ) -> bool:
461         if self.system.encode_formula(certificate.conclusion) != certificate.conclusion_line:
462             return False
463
464         if certificate.rule_name == self.AXIOM_RULE:
465             return (
466                 certificate.conclusion in self._axioms
467                 and not certificate.premise_lines
468                 and not certificate.premise_formulas
469             )
470
471         try:
472             rule = self.system.rule_by_name(certificate.rule_name)
473         except KeyError:
474             return False
475
476         if len(certificate.premise_lines) != rule.arity:
477             return False
478         if len(certificate.premise_formulas) != rule.arity:
479             return False
480
481         recovered: list[FormulaT] = []
482         for line, claimed_formula in zip(
483             certificate.premise_lines, certificate.premise_formulas, strict=True
484         ):
485             record = known_by_line.get(line)
486             if record is None or record.formula != claimed_formula:
487                 return False
488             recovered.append(record.formula)

```



```

489         produced = tuple(rule.derive(tuple(recovered)))
490         return certificate.conclusion in produced
491
492
493
494 class TheoremDatabase(Generic[FormulaT]):
495     """Monotone verified theorem store with unique WFF line numbers."""
496
497     def __init__(
498         self,
499         system: FormalSystem[FormulaT],
500         verifier: IndependentVerifier[FormulaT],
501     ) -> None:
502         self.system = system
503         self.verifier = verifier
504         self._by_formula: dict[FormulaT, TheoremRecord[FormulaT]] = {}
505         self._by_line: dict[int, TheoremRecord[FormulaT]] = {}
506         self._load_axioms()
507
508     def _load_axioms(self) -> None:
509         for axiom in self.system.axioms:
510             line = self.system.encode_formula(axiom)
511             certificate = ProofCertificate(
512                 conclusion=axiom,
513                 conclusion_line=line,
514                 rule_name=IndependentVerifier.AXIOM_RULE,
515                 premise_lines=(),
516                 premise_formulas=(),
517                 inference_line=None,
518             )
519             record = TheoremRecord(axiom, line, certificate, discovery_epoch=0)
520             if not self.verifier.verify(certificate, self._by_line):
521                 raise ValueError(f"axiom failed verification: {axiom!r}")
522             self._insert(record)
523
524     def _insert(self, record: TheoremRecord[FormulaT]) -> None:
525         previous_line = self._by_line.get(record.wff_line)
526         if previous_line is not None and previous_line.formula != record.formula:
527             raise ValueError(
528                 f"formula encoder collision at line {record.wff_line}: "
529                 f"{previous_line.formula!r} vs {record.formula!r}"
530             )
531         self._by_formula[record.formula] = record
532         self._by_line[record.wff_line] = record
533
534     def contains(self, formula: FormulaT) -> bool:
535         return formula in self._by_formula
536
537     def record_for_formula(self, formula: FormulaT) -> TheoremRecord[FormulaT]:
538         return self._by_formula[formula]
539
540     def known_by_line(self) -> Mapping[int, TheoremRecord[FormulaT]]:
541         return self._by_line
542
543     def frozen_records(self) -> tuple[TheoremRecord[FormulaT], ...]:
544         return tuple(sorted(self._by_formula.values(), key=lambda item: item.wff_line))
545
546     def commit_staged(
547         self,
548         staged: Mapping[FormulaT, ProofCertificate[FormulaT]],
549         epoch: int,
550     ) -> tuple[TheoremRecord[FormulaT], ...]:
551         accepted: list[TheoremRecord[FormulaT]] = []
552         for formula in sorted(staged, key=self.system.canonical_key):
553             if self.contains(formula):
554                 continue
555             certificate = staged[formula]
556             if not self.verifier.verify(certificate, self._by_line):
557                 raise ValueError(f"staged certificate failed verification: {formula!r}")
558             record = TheoremRecord(
559                 formula=formula,

```

```

560         wff_line=certificate.conclusion_line,
561         certificate=certificate,
562         discovery_epoch=epoch,
563     )
564     self._insert(record)
565     accepted.append(record)
566     return tuple(accepted)
567
568     def __len__(self) -> int:
569         return len(self._by_formula)
570
571
572     # -----
573     # Part IV. Auditable budgets, scheduling, and frozen Hilbert epochs
574     # -----
575
576
577 @dataclass(slots=True)
578 class Budget:
579     expansions_remaining: int
580     address_visits_remaining: int
581     verifier_calls_remaining: int
582
583     def __post_init__(self) -> None:
584         if min(
585             self.expansions_remaining,
586             self.address_visits_remaining,
587             self.verifier_calls_remaining,
588         ) < 0:
589             raise ValueError("budget entries must be nonnegative")
590
591
592 @dataclass(frozen=True, slots=True)
593 class RuleCoverage:
594     rule_name: str
595     arity: int
596     hilbert_bits: int
597     cells_total: int
598     cells_visited: int
599     valid_tuples_attempted: int
600     padding_cells_visited: int
601     verified_novel_candidates: int
602     complete: bool
603
604
605 @dataclass(frozen=True, slots=True)
606 class EpochReport(Generic[FormulaT]):
607     epoch: int
608     frozen_theorem_count: int
609     ambient_dimension: int
610     expansions_used: int
611     address_visits_used: int
612     verifier_calls_used: int
613     new_theorems: tuple[TheoremRecord[FormulaT], ...]
614     rule_coverage: tuple[RuleCoverage, ...]
615     complete: bool
616     log_chain_valid: bool
617
618
619 class RuleScheduler(Protocol[FormulaT]):
620     def choose_rule(
621         self,
622         rules: Sequence[InferenceRule[FormulaT]],
623         cursors: Mapping[str, HilbertCursor],
624     ) -> InferenceRule[FormulaT] | None:
625         """Return an unfinished positive-arity rule, or None if all are done."""
626
627
628 @dataclass(slots=True)
629 class RoundRobinScheduler(Generic[FormulaT]):
630     """Deterministic fair scheduler; a learned policy can replace this class."""

```

```

631
632     next_index: int = 0
633
634     def choose_rule(
635         self,
636         rules: Sequence[InferenceRule[FormulaT]],
637         cursors: Mapping[str, HilbertCursor],
638     ) -> InferenceRule[FormulaT] | None:
639         eligible = [rule for rule in rules if rule.arity > 0 and not cursors[rule.name].done]
640         if not eligible:
641             return None
642         rule = eligible[self.next_index % len(eligible)]
643         self.next_index += 1
644         return rule
645
646
647 class HashChainLog:
648     """Small append-only SHA-256 transaction log."""
649
650     def __init__(self) -> None:
651         self.entries: list[dict[str, object]] = []
652
653     def append(self, event: str, payload: Mapping[str, object]) -> None:
654         previous = self.entries[-1]["digest"] if self.entries else "GENESIS"
655         sequence = len(self.entries)
656         canonical = json.dumps(
657             {
658                 "sequence": sequence,
659                 "event": event,
660                 "payload": payload,
661                 "previous": previous,
662             },
663             sort_keys=True,
664             separators=(",", ":"),
665             default=str,
666         )
667         digest = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
668         self.entries.append(
669             {
670                 "sequence": sequence,
671                 "event": event,
672                 "payload": dict(payload),
673                 "previous": previous,
674                 "digest": digest,
675             }
676         )
677
678     def verify(self) -> bool:
679         previous = "GENESIS"
680         for sequence, entry in enumerate(self.entries):
681             if entry["sequence"] != sequence or entry["previous"] != previous:
682                 return False
683             canonical = json.dumps(
684                 {
685                     "sequence": sequence,
686                     "event": entry["event"],
687                     "payload": entry["payload"],
688                     "previous": entry["previous"],
689                 },
690                 sort_keys=True,
691                 separators=(",", ":"),
692                 default=str,
693             )
694             expected = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
695             if entry["digest"] != expected:
696                 return False
697             previous = expected
698         return True
699
700
701 @dataclass(slots=True)

```

```

702 class _MutableCoverage:
703     rule_name: str
704     arity: int
705     hilbert_bits: int
706     cells_total: int
707     cells_visited: int = 0
708     valid_tuples_attempted: int = 0
709     padding_cells_visited: int = 0
710     verified_novel_candidates: int = 0
711
712     def freeze(self, complete: bool) -> RuleCoverage:
713         return RuleCoverage(
714             rule_name=self.rule_name,
715             arity=self.arity,
716             hilbert_bits=self.hilbert_bits,
717             cells_total=self.cells_total,
718             cells_visited=self.cells_visited,
719             valid_tuples_attempted=self.valid_tuples_attempted,
720             padding_cells_visited=self.padding_cells_visited,
721             verified_novel_candidates=self.verified_novel_candidates,
722             complete=complete,
723         )
724
725 class HilbertTheoremSearch(Generic[FormulaT]):
726     """Budgeted theorem-discovery engine over a coproduct of rule cubes."""
727
728     def __init__(
729         self,
730         system: FormalSystem[FormulaT],
731         scheduler: RuleScheduler[FormulaT] | None = None,
732     ) -> None:
733         self.system = system
734         self.verifier = IndependentVerifier(system)
735         self.database = TheoremDatabase(system, self.verifier)
736         self.scheduler = scheduler or RoundRobinScheduler()
737         self.log = HashChainLog()
738         self.epoch = 0
739
740     def _certificate(
741         self,
742         rule_tag: int,
743         rule: InferenceRule[FormulaT],
744         premises: tuple[TheoremRecord[FormulaT], ...],
745         conclusion: FormulaT,
746     ) -> ProofCertificate[FormulaT]:
747         lines = tuple(record.wff_line for record in premises)
748         return ProofCertificate(
749             conclusion=conclusion,
750             conclusion_line=self.system.encode_formula(conclusion),
751             rule_name=rule.name,
752             premise_lines=lines,
753             premise_formulas=tuple(record.formula for record in premises),
754             inference_line=inference_address_line(rule_tag, lines),
755         )
756
757     def run_epoch(self, budget: Budget) -> EpochReport[FormulaT]:
758         """Run one frozen theorem-discovery epoch.
759
760         New conclusions are staged and become usable as premises only after the
761         epoch ends. A complete epoch therefore computes exactly one direct-
762         consequence layer relative to the frozen theorem set.
763         """
764         self.epoch += 1
765         epoch = self.epoch
766         frozen = self.database.frozen_records()
767         theorem_count = len(frozen)
768         bits = max(0, (theorem_count - 1).bit_length())
769
770         positive_rules = [rule for rule in self.system.rules if rule.arity > 0]
771         cursors = {

```

```

773         rule.name: HilbertCursor(rule.arity, bits) for rule in positive_rules
774     }
775     coverage = {
776         rule.name: _MutableCoverage(
777             rule_name=rule.name,
778             arity=rule.arity,
779             hilbert_bits=bits,
780             cells_total=cursors[rule.name].total_cells,
781         )
782         for rule in positive_rules
783     }
784     staged: dict[FormulaT, ProofCertificate[FormulaT]] = {}
785
786     start_expansions = budget.expansions_remaining
787     start_visits = budget.address_visits_remaining
788     start_verifier = budget.verifier_calls_remaining
789
790     self.log.append(
791         "EpochStarted",
792         {
793             "epoch": epoch,
794             "frozen_theorems": theorem_count,
795             "hilbert_bits": bits,
796             "ambient_dimension": self.system.ambient_orthogonal_dimension,
797         },
798     )
799
800     # Zero-arity rules are evaluated once per epoch.
801     zero_arity_complete = True
802     for rule_tag, rule in enumerate(self.system.rules):
803         if rule.arity != 0:
804             continue
805         if budget.expansions_remaining <= 0 or budget.verifier_calls_remaining <= 0:
806             zero_arity_complete = False
807             break
808         budget.expansions_remaining -= 1
809         for conclusion in rule.derive():
810             if self.database.contains(conclusion) or conclusion in staged:
811                 continue
812             certificate = self._certificate(rule_tag, rule, (), conclusion)
813             budget.verifier_calls_remaining -= 1
814             if self.verifier.verify(certificate, self.database.known_by_line()):
815                 staged[conclusion] = certificate
816
817     rule_tag_by_name = {
818         rule.name: tag for tag, rule in enumerate(self.system.rules)
819     }
820
821     while (
822         budget.expansions_remaining > 0
823         and budget.address_visits_remaining > 0
824         and budget.verifier_calls_remaining > 0
825     ):
826         rule = self.scheduler.choose_rule(positive_rules, cursors)
827         if rule is None:
828             break
829
830         cursor = cursors[rule.name]
831         point = cursor.next_point()
832         stat = coverage[rule.name]
833         stat.cells_visited += 1
834         budget.address_visits_remaining -= 1
835
836         self.log.append(
837             "HilbertCellVisited",
838             {
839                 "epoch": epoch,
840                 "rule": rule.name,
841                 "distance": cursor.next_distance - 1,
842                 "point": point,
843             },

```

```

844         )
845
846         if any(index >= theorem_count for index in point):
847             stat.padding_cells_visited += 1
848             continue
849
850         premise_records = tuple(frozen[index] for index in point)
851         stat.valid_tuples_attempted += 1
852         budget.expansions_remaining -= 1
853
854         self.log.append(
855             "ExpansionAttempted",
856             {
857                 "epoch": epoch,
858                 "rule": rule.name,
859                 "premise_lines": tuple(item.wff_line for item in premise_records),
860             },
861         )
862
863         conclusions = tuple(rule.derive(tuple(item.formula for item in premise_records)))
864         for conclusion in conclusions:
865             if self.database.contains(conclusion) or conclusion in staged:
866                 continue
867             if budget.verifier_calls_remaining <= 0:
868                 break
869             certificate = self._certificate(
870                 rule_tag_by_name[rule.name], rule, premise_records, conclusion
871             )
872             budget.verifier_calls_remaining -= 1
873             accepted = self.verifier.verify(
874                 certificate, self.database.known_by_line()
875             )
876             self.log.append(
877                 "CandidateVerified",
878                 {
879                     "epoch": epoch,
880                     "rule": rule.name,
881                     "conclusion_line": certificate.conclusion_line,
882                     "accepted": accepted,
883                     "inference_line": certificate.inference_line,
884                 },
885             )
886             if accepted:
887                 staged[conclusion] = certificate
888                 stat.verified_novel_candidates += 1
889
890         committed = self.database.commit_staged(staged, epoch)
891         all_positive_complete = all(cursor.done for cursor in cursors.values())
892         complete = all_positive_complete and zero_arity_complete
893
894         self.log.append(
895             "EpochFinished",
896             {
897                 "epoch": epoch,
898                 "complete": complete,
899                 "new_theorems": tuple(record.wff_line for record in committed),
900             },
901         )
902
903         return EpochReport(
904             epoch=epoch,
905             frozen_theorem_count=theorem_count,
906             ambient_dimension=self.system.ambient_orthogonal_dimension,
907             expansions_used=start_expansions - budget.expansions_remaining,
908             address_visits_used=start_visits - budget.address_visits_remaining,
909             verifier_calls_used=start_verifier - budget.verifier_calls_remaining,
910             new_theorems=committed,
911             rule_coverage=tuple(
912                 coverage[rule.name].freeze(cursors[rule.name].done)
913                 for rule in positive_rules
914             ),

```

```

915         complete=complete,
916         log_chain_valid=self.log.verify(),
917     )
918
919     def discover(
920         self,
921         budget: Budget,
922         max_epochs: int | None = None,
923         stop_when_saturated: bool = True,
924     ) -> tuple[EpochReport[FormulaT], ...]:
925         reports: list[EpochReport[FormulaT]] = []
926         while (
927             budget.expansions_remaining > 0
928             and budget.address_visits_remaining > 0
929             and budget.verifier_calls_remaining > 0
930             and (max_epochs is None or len(reports) < max_epochs)
931         ):
932             report = self.run_epoch(budget)
933             reports.append(report)
934             if stop_when_saturated and report.complete and not report.new_theorems:
935                 break
936             if not report.complete:
937                 break
938         return tuple(reports)
939
940
941 # -----
942 # Part V. Exhaustive baseline and an executable toy formal system
943 # -----
944
945
946 @dataclass(frozen=True, order=True, slots=True)
947 class ToyFormula:
948     """Toy WFF P(n), represented by a bounded nonnegative integer n."""
949
950     value: int
951
952     def __str__(self) -> str:
953         return f"P({self.value})"
954
955
956 def make_toy_system(max_value: int = 12) -> FormalSystem[ToyFormula]:
957     """Create a finite system with unary, binary, and ternary rules.
958
959     Axioms: P(0), P(1)
960     succ:   P(a) -> P(a+1)
961     add:    P(a), P(b) -> P(a+b)
962     affine: P(a), P(b), P(c) -> P(a+2b+3c)
963
964     Conclusions above max_value are undefined and produce no formula.
965     """
966     if max_value < 1:
967         raise ValueError("max_value must be at least 1")
968
969     def bounded(value: int) -> tuple[ToyFormula, ...]:
970         return (ToyFormula(value),) if 0 <= value <= max_value else ()
971
972     def succ(premises: tuple[ToyFormula, ...]) -> Iterable[ToyFormula]:
973         (a,) = premises
974         return bounded(a.value + 1)
975
976     def add(premises: tuple[ToyFormula, ...]) -> Iterable[ToyFormula]:
977         a, b = premises
978         return bounded(a.value + b.value)
979
980     def affine(premises: tuple[ToyFormula, ...]) -> Iterable[ToyFormula]:
981         a, b, c = premises
982         return bounded(a.value + 2 * b.value + 3 * c.value)
983
984     return FormalSystem(
985         name=f"Bounded toy arithmetic through {max_value}",

```

```

986     axioms=(ToyFormula(0), ToyFormula(1)),
987     rules=(
988         InferenceRule("succ", 1, succ),
989         InferenceRule("add", 2, add),
990         InferenceRule("affine", 3, affine),
991     ),
992     encode_formula=lambda formula: formula.value,
993     canonical_key=lambda formula: formula.value,
994 )
995
996
997 def exhaustive_direct_consequences(
998     system: FormalSystem[FormulaT],
999     formulas: Sequence[FormulaT],
1000 ) -> set[FormulaT]:
1001     """Reference implementation of one direct-consequence layer."""
1002     import itertools
1003
1004     result = set(formulas)
1005     for rule in system.rules:
1006         if rule.arity == 0:
1007             tuples: Iterable[tuple[FormulaT, ...]] = [()]
1008         else:
1009             tuples = itertools.product(formulas, repeat=rule.arity)
1010         for premises in tuples:
1011             result.update(rule.derive(tuple(premises)))
1012     return result
1013
1014
1015 def demo() -> None:
1016     system = make_toy_system(max_value=12)
1017     search = HilbertTheoremSearch(system)
1018     budget = Budget(
1019         expansions_remaining=20_000,
1020         address_visits_remaining=30_000,
1021         verifier_calls_remaining=20_000,
1022     )
1023     reports = search.discover(budget, max_epochs=8)
1024
1025     print("Hilbert Space Filling Curve-Style Theorem Search")
1026     print("By ChatGPT, inspired by Brian Tenneson")
1027     print(f"Formal system: {system.name}")
1028     print(f"Ambient orthogonal dimension K = {system.ambient_orthogonal_dimension}")
1029     print()
1030
1031     for report in reports:
1032         print(
1033             f"epoch={report.epoch} frozen={report.frozen_theorem_count} "
1034             f"expansions={report.expansions_used} "
1035             f"new={len(report.new_theorems)} complete={report.complete}"
1036         )
1037         for item in report.rule_coverage:
1038             print(
1039                 f" {item.rule_name:<8} arity={item.arity} "
1040                 f"cells={item.cells_visited}/{item.cells_total} "
1041                 f"tuples={item.valid_tuples_attempted} "
1042                 f"padding={item.padding_cells_visited}"
1043             )
1044
1045     formulas = [record.formula for record in search.database.frozen_records()]
1046     print()
1047     print("Verified theorem database:")
1048     print("", ".join(str(formula) for formula in formulas))
1049     print("Hash chain valid:", search.log.verify())
1050
1051
1052 if __name__ == "__main__":
1053     demo()

```


Appendix B. Automated tests

```

1  from fractions import Fraction
2  import itertools
3  import unittest
4
5  from hilbert_theorem_search import (
6      Budget,
7      HilbertCurve,
8      HilbertTheoremSearch,
9      ProofCertificate,
10     ToyFormula,
11     a002260,
12     a004736,
13     exhaustive_direct_consequences,
14     inference_address_line,
15     make_toy_system,
16     pair_positive,
17     pair_tuple,
18     premise_point,
19     rational_cube_line,
20     rational_cube_point,
21     tenneson_r,
22     tenneson_s,
23     unit_rational,
24     unit_rational_section,
25     unpair_positive,
26     unpair_tuple,
27     wff_coordinate,
28     wff_line_from_coordinate,
29 )
30
31
32 class TennesonAddressTests(unittest.TestCase):
33     def test_sequence_coordinates(self):
34         self.assertEqual([a002260(n) for n in range(1, 11)], [1,1,2,1,2,3,1,2,3,4])
35         self.assertEqual([a004736(n) for n in range(1, 11)], [1,2,1,3,2,1,4,3,2,1])
36
37     def test_r_and_s(self):
38         samples = [
39             Fraction(0), Fraction(1), Fraction(-1), Fraction(2,3),
40             Fraction(-7,5), Fraction(13,11), Fraction(-19,23),
41         ]
42         for q in samples:
43             self.assertEqual(tenneson_r(tenneson_s(q)), q)
44             least = tenneson_s(q)
45             self.assertTrue(all(tenneson_r(n) != q for n in range(1, least)))
46
47     def test_pairing(self):
48         for a in range(1, 20):
49             for b in range(1, 20):
50                 self.assertEqual(unpair_positive(pair_positive(a,b)), (a,b))
51         values = (3, 7, 2, 9)
52         self.assertEqual(unpair_tuple(pair_tuple(values), len(values)), values)
53
54     def test_unit_interval_section(self):
55         samples = [Fraction(0), Fraction(1), Fraction(1,2), Fraction(2,3), Fraction(17,19)]
56         for q in samples:
57             line = unit_rational_section(q)
58             self.assertEqual(unit_rational(line), q)
59             self.assertTrue(all(unit_rational(n) != q for n in range(1, line)))
60
61     def test_rational_cube_round_trip(self):
62         point = (Fraction(0), Fraction(1,2), Fraction(7,9), Fraction(1))
63         line = rational_cube_line(point)
64         self.assertEqual(rational_cube_point(line, len(point)), point)
65
66     def test_wff_coordinates_are_injective(self):
67         coords = [wff_coordinate(g) for g in range(100)]

```

```

68         self.assertEqual(len(coords), len(set(coords)))
69         for g, q in enumerate(coords):
70             self.assertEqual(wff_line_from_coordinate(q), g)
71
72
73     class HilbertTests(unittest.TestCase):
74         def test_bijection_and_adjacency(self):
75             for dimension in range(1, 6):
76                 for bits in range(0, 4):
77                     curve = HilbertCurve(dimension, bits)
78                     points = list(curve.points())
79                     self.assertEqual(len(points), curve.size)
80                     self.assertEqual(len(set(points)), curve.size)
81                     for distance, point in enumerate(points):
82                         self.assertEqual(curve.distance_from_point(point), distance)
83                     for left, right in zip(points, points[1:]):
84                         manhattan = sum(abs(a-b) for a,b in zip(left,right))
85                         self.assertEqual(manhattan, 1)
86
87
88     class SearchTests(unittest.TestCase):
89         def test_complete_epoch_matches_exhaustive_layer(self):
90             system = make_toy_system(max_value=8)
91             search = HilbertTheoremSearch(system)
92             frozen = [record.formula for record in search.database.frozen_records()]
93             expected = exhaustive_direct_consequences(system, frozen)
94             report = search.run_epoch(Budget(1000, 1000, 1000))
95             observed = {record.formula for record in search.database.frozen_records()}
96             self.assertTrue(report.complete)
97             self.assertEqual(observed, expected)
98             expected_expansions = sum(len(frozen) * rule.arity for rule in system.rules)
99             self.assertEqual(report.expansions_used, expected_expansions)
100
101         def test_every_valid_tuple_once(self):
102             system = make_toy_system(max_value=5)
103             search = HilbertTheoremSearch(system)
104             m = len(search.database)
105             report = search.run_epoch(Budget(1000, 1000, 1000))
106             for coverage, rule in zip(report.rule_coverage, system.rules):
107                 self.assertEqual(coverage.valid_tuples_attempted, m * rule.arity)
108                 self.assertEqual(
109                     coverage.cells_visited,
110                     coverage.valid_tuples_attempted + coverage.padding_cells_visited,
111                 )
112
113         def test_partial_budget_is_reported(self):
114             system = make_toy_system(max_value=8)
115             search = HilbertTheoremSearch(system)
116             report = search.run_epoch(Budget(3, 100, 100))
117             self.assertFalse(report.complete)
118             self.assertEqual(report.expansions_used, 3)
119             self.assertTrue(report.log_chain_valid)
120
121         def test_tampered_certificate_is_rejected(self):
122             system = make_toy_system(max_value=5)
123             search = HilbertTheoremSearch(system)
124             known = search.database.known_by_line()
125             bad = ProofCertificate(
126                 conclusion=ToyFormula(5),
127                 conclusion_line=5,
128                 rule_name="succ",
129                 premise_lines=(0,),
130                 premise_formulas=(ToyFormula(0),),
131                 inference_line=inference_address_line(0, (0,)),
132             )
133             self.assertFalse(search.verifier.verify(bad, known))
134
135         def test_discovery_reaches_finite_closure(self):
136             system = make_toy_system(max_value=10)
137             search = HilbertTheoremSearch(system)
138             reports = search.discover(Budget(50000, 100000, 50000), max_epochs=10)

```

```
139         values = {record.formula.value for record in search.database.frozen_records()}
140         self.assertEqual(values, set(range(11)))
141         self.assertTrue(any(report.complete and not report.new_theorems for report in reports))
142
143
144 if __name__ == "__main__":
145     unittest.main()
```

Appendix C. Build, verification, and stress-test note

Version 0.4.0 document verification. Version 0.4.0 retains the Version 0.3.0 theorem-search implementation and its 12-test unittest result; the code is not changed by this pagination-only revision. The source now explicitly restores page 2 immediately after the unnumbered title page so the TOC, footer page numbers, LastPage total, and physical PDF pages use one consistent numbering scheme. Version 0.2.0 had additionally stress-checked Hilbert bijection, inverse mapping, and face adjacency through representative grids in dimension 6; Tenneson's r/s least-preimage behavior across the first 5,000 enumerated rational outputs; and the derived unit-interval section on reduced rational grids with denominators below 30. Those checks increase confidence in the reference algorithms but are not a formal proof of implementation correctness and are not a substitute for a production proof kernel.

Compilation record

The source file `main.tex` was compiled with XeLaTeX in the build environment. The final PDF was then rendered to page images for layout inspection. The deliverable therefore has both a machine-checked LaTeX compilation path and a render-level verification step.

Section X

This final section is deliberately exploratory. The conjectures below are falsifiable research hypotheses, not established facts. The theorems that follow are statements that can be justified directly from the definitions used in this paper; they are not claimed to be deep or historically unprecedented results. They are included because they make the proposed architecture easier to test, criticize, and implement.

Conjectures

Conjecture 1 (Semantic-locality productivity). *Let a fixed formal library be equipped with a stable formula embedding whose local neighborhoods correlate positively with proof relevance. Under a fixed expansion budget, a Hilbert traversal of the induced premise-coordinate space will yield more verified novel theorems, or a shorter time to a specified target proof, than lexicographic ordering and uniformly random ordering on a nontrivial fraction of benchmark problems.*

The conjecture is intentionally conditional on the embedding. The present WFF-line ordering need not have the required semantic locality, so a negative result for that ordering would not by itself refute the broader hypothesis.

Conjecture 2 (Interruption robustness). *Suppose useful inference addresses occur in spatially clustered regions of a fixed coordinate representation and the stopping time is unknown in advance. A coarse-to-fine Hilbert schedule will have lower expected uncovered-region radius at interruption, and consequently more stable verified-theorem yield, than a one-direction lexicographic sweep with the same number of address visits.*

This is the proof-search analogue of the coverage motivation that led to the present construction. It should be tested with both synthetic clustered landscapes and real formal libraries.

Conjecture 3 (Fair learned coproduct scheduling). *For heterogeneous rule arities, a scheduler that combines a fair Hilbert fallback with learned estimates of marginal theorem yield per expansion will outperform uniform round-robin allocation among coproduct components while preserving eventual completeness whenever the fallback is allowed to run to completion.*

The important restriction is fairness: learning may change the order and budget allocation, but a run advertised as exhaustive must not permanently starve a rule component.

Conjecture 4 (The `sgrpcl` pilot). *In the frozen Predator/Metamath `sgrpcl` environment, a legal-first Hilbert-aware ordering of canonically addressed assertion applications will require fewer expansions to the first externally verified `sgrpcl` certificate than the matched dense-uniform legal-first control, conditional on both methods producing a certificate within the same declared budget.*

This conjecture is deliberately narrow. It is about one frozen target and one matched comparison; failure would be scientifically useful and would not imply that all Hilbert-style proof-space orderings are ineffective.

Theorems

Theorem 1 (Canonical rational premise addressing). *Define*

$$\iota(g) = \frac{g}{g+1}, \quad g \in \mathbb{N},$$

and for $k \geq 1$ define

$$I_k(g_1, \dots, g_k) = (\iota(g_1), \dots, \iota(g_k)).$$

Then I_k is injective from $\mathbb{N}^k$ into $(\mathbb{Q} \cap [0, 1))^k$. Moreover, if

$$R_k : \mathbb{N}^+ \rightarrow (\mathbb{Q} \cap [0, 1))^k$$

has a right inverse S_k , then

$$L_k = S_k \circ I_k : \mathbb{N}^k \rightarrow \mathbb{N}^+$$

is injective. Thus every ordered k -tuple of WFF line numbers has a unique canonical natural-number address under this construction.

Proof. The function ι is injective because

$$\frac{a}{a+1} = \frac{b}{b+1}$$

implies $a(b+1) = b(a+1)$ and hence $a = b$. Coordinatewise application therefore makes I_k injective. If $L_k(g) = L_k(h)$, applying R_k to both sides gives

$$I_k(g) = R_k(S_k(I_k(g))) = R_k(S_k(I_k(h))) = I_k(h),$$

and injectivity of I_k yields $g = h$. □

Theorem 2 (Coproduct candidate count). Let C be a finite theorem set with $|C| = M$, and let the formal system have rules of arities $k_1, \dots, k_m$. The number of tagged ordered premise tuples in the intrinsic action space

$$\mathcal{A}_F(C) = \bigsqcup_{i=1}^m C^{k_i}$$

is exactly

$$|\mathcal{A}_F(C)| = \sum_{i=1}^m M^{k_i}.$$

By contrast, the Cartesian product of all rule-coordinate blocks has size

$$M^{\sum_i k_i}.$$

Proof. The i th tagged component contains exactly M^{k_i} ordered tuples. Coproduct tags make distinct components disjoint, so cardinalities add. The full Cartesian product is naturally isomorphic to $C^{k_1 + \dots + k_m}$, whose cardinality is $M^{\sum_i k_i}$. □

Theorem 3 (Exact valid-tuple coverage inside a padded Hilbert grid). Let $M \geq 1$, let $N = 2^{\lceil \log_2 M \rceil}$, and let

$$H : \{0, \dots, N^k - 1\} \longrightarrow \{0, \dots, N - 1\}^k$$

be a bijective discrete k -dimensional Hilbert traversal. Mark a cell valid when every coordinate is less than M . Then every element of $\{0, \dots, M - 1\}^k$ is visited exactly once, and no padding cell duplicates a valid tuple.

Proof. The valid grid is a subset of the Hilbert traversal's codomain. Since H is a bijection onto the entire padded grid, every grid point has exactly one preimage. Restricting to the valid subset preserves that one-to-one visitation property. □

Theorem 4 (Frozen-epoch equivalence). Let C be a finite verified theorem set and let each rule r_i be an effective finite-arity partial operation on explicit ordered premises. Suppose an epoch attempts every tuple in every component C^{k_i} exactly once, stages every legal novel conclusion, and the verification layer accepts exactly the legal rule consequences. Then the committed theorem set at the end of the epoch is exactly

$$D_F(C) = C \cup \{r_i(a) : a \in C^{k_i}, r_i(a) \text{ is defined}\}.$$

Proof. Every committed novel formula is, by hypothesis, the verified output of some rule applied to a tuple from C , so the committed set is contained in $D_F(C)$. Conversely, every defined rule consequence in $D_F(C)$ comes from a tuple that the completed epoch attempts; exact verification accepts it, so it is either already in C or is staged and committed. Hence equality holds. $\square$

Theorem 5 (Positive enumeration by complete epochs). *Let $C_0 = \Gamma$ and define $C_{e+1} = D_F(C_e)$ under the hypotheses of the preceding theorem. If every epoch completes, then*

$$\bigcup_{e=0}^{\infty} C_e = \text{Con}_F(\Gamma),$$

where $\text{Con}_F(\Gamma)$ is the set of formulas having finite derivations from Γ using the rules of F .

Proof. For the forward inclusion, every member of C_0 is in Γ , and every later member is obtained by one legal inference from formulas already possessing finite derivations, so every member of the union has a finite derivation. For the reverse inclusion, induct on the length of a finite derivation of φ . The base case lies in C_0 . If the last step applies r_i to premises whose derivations are shorter, the induction hypothesis places all premises in some common C_e because the sequence is monotone. The next complete epoch applies r_i to that tuple, so $\varphi \in C_{e+1}$. Therefore every finitely derivable formula appears in the union. $\square$

Interpretation. The final theorem is a positive-enumeration statement, not a decision theorem. It guarantees eventual discovery of finitely provable consequences under the stated fairness and effectiveness assumptions; it does not give a finite stage at which failure to discover a formula becomes a correct declaration of non-theoremhood.